

-Kabir Sinha

-E23CSEU1343

CSET419 – Introduction to Generative AI

Lab 2: Generative Adversarial Network (GAN) for Synthetic Image Generation using MNIST Dataset

```
[10]
✓ Os
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image
import matplotlib.pyplot as plt
import os

[11]
✓ Os
# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Hyperparameters
latent_size = 64
hidden_size = 256
image_size = 784
num_epochs = 30
batch_size = 100
learning_rate = 0.0002

# Create directory for generated images
sample_dir = 'samples'
os.makedirs(sample_dir, exist_ok=True)
```

```
[12]
✓ Os
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.5,), std=(0.5,))
])

mnist = torchvision.datasets.MNIST(root='./data',
                                   train=True,
                                   transform=transform,
                                   download=True)

data_loader = torch.utils.data.DataLoader(dataset=mnist,
                                           batch_size=batch_size,
                                           shuffle=True)

[13]
✓ Os
D = nn.Sequential(
    nn.Linear(image_size, hidden_size),
    nn.LeakyReLU(0.2),
    nn.Linear(hidden_size, hidden_size),
    nn.LeakyReLU(0.2),
    nn.Linear(hidden_size, 1),
    nn.Sigmoid()
).to(device)

[14]
✓ Os
G = nn.Sequential(
    nn.Linear(latent_size, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, hidden_size),
    nn.ReLU(),
    nn.Linear(hidden_size, image_size),
    nn.Tanh()
).to(device)
```

```
[15]
✓ 0s criterion = nn.BCELoss()

d_optimizer = torch.optim.Adam(D.parameters(), lr=learning_rate)
g_optimizer = torch.optim.Adam(G.parameters(), lr=learning_rate)
```

```
[16]
✓ 6m total_step = len(data_loader)

for epoch in range(num_epochs):
    for i, (images, _) in enumerate(data_loader):

        images = images.reshape(batch_size, -1).to(device)

        real_labels = torch.ones(batch_size, 1).to(device)
        fake_labels = torch.zeros(batch_size, 1).to(device)

        # Train Discriminator
        outputs = D(images)
        d_loss_real = criterion(outputs, real_labels)

        z = torch.randn(batch_size, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images.detach())
        d_loss_fake = criterion(outputs, fake_labels)

        d_loss = d_loss_real + d_loss_fake

        d_optimizer.zero_grad()
        g_optimizer.zero_grad()
        d_loss.backward()
        d_optimizer.step()

        # Train Generator
        z = torch.randn(batch_size, latent_size).to(device)
        fake_images = G(z)
        outputs = D(fake_images)
        g_loss = criterion(outputs, real_labels)
```

```

d_optimizer.zero_grad()
g_optimizer.zero_grad()
g_loss.backward()
g_optimizer.step()

if (i+1) % 200 == 0:
    print(f"Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{total_step}], "
          f"d_loss: {d_loss.item():.4f}, g_loss: {g_loss.item():.4f}")

# Save generated images
fake_images = fake_images.reshape(fake_images.size(0), 1, 28, 28)
save_image(fake_images, os.path.join(sample_dir, f'fake_images-{epoch+1}.png'))
```

Epoch [1/30], Step [200/600], d_loss: 0.0315, g_loss: 4.6497

Epoch [1/30], Step [400/600], d_loss: 0.2469, g_loss: 4.6398
Epoch [1/30], Step [600/600], d_loss: 0.0299, g_loss: 5.6350
Epoch [2/30], Step [200/600], d_loss: 0.3114, g_loss: 4.6266
Epoch [2/30], Step [400/600], d_loss: 0.1136, g_loss: 4.5197
Epoch [2/30], Step [600/600], d_loss: 0.7550, g_loss: 4.5219
Epoch [3/30], Step [200/600], d_loss: 0.3578, g_loss: 3.6721
Epoch [3/30], Step [400/600], d_loss: 0.2544, g_loss: 5.3729
Epoch [3/30], Step [600/600], d_loss: 0.9493, g_loss: 2.4941
Epoch [4/30], Step [200/600], d_loss: 0.4390, g_loss: 1.9264
Epoch [4/30], Step [400/600], d_loss: 0.2685, g_loss: 3.4954
Epoch [4/30], Step [600/600], d_loss: 0.4966, g_loss: 3.5625
Epoch [5/30], Step [200/600], d_loss: 0.5177, g_loss: 2.3481
Epoch [5/30], Step [400/600], d_loss: 0.2570, g_loss: 2.6925
Epoch [5/30], Step [600/600], d_loss: 0.2412, g_loss: 3.4401
Epoch [6/30], Step [200/600], d_loss: 0.9713, g_loss: 3.3076
Epoch [6/30], Step [400/600], d_loss: 0.2815, g_loss: 3.2134
Epoch [6/30], Step [600/600], d_loss: 0.2166, g_loss: 3.9528
Epoch [7/30], Step [200/600], d_loss: 0.3463, g_loss: 4.4033
Epoch [7/30], Step [400/600], d_loss: 0.2715, g_loss: 3.6068
Epoch [7/30], Step [600/600], d_loss: 0.6649, g_loss: 3.8198
Epoch [8/30], Step [200/600], d_loss: 0.2716, g_loss: 4.1080
Epoch [8/30], Step [400/600], d_loss: 0.0720, g_loss: 3.5842
Epoch [8/30], Step [600/600], d_loss: 0.0544, g_loss: 4.2156
Epoch [9/30], Step [200/600], d_loss: 0.1654, g_loss: 3.5453
Epoch [9/30], Step [400/600], d_loss: 0.1840, g_loss: 4.0802
Epoch [9/30], Step [600/600], d_loss: 0.1840, g_loss: 4.5644
Epoch [10/30], Step [200/600], d_loss: 0.2297, g_loss: 4.2874
Epoch [10/30], Step [400/600], d_loss: 0.2301, g_loss: 4.4869
Epoch [10/30], Step [600/600], d_loss: 0.3076, g_loss: 3.6039
Epoch [11/30], Step [200/600], d_loss: 0.1994, g_loss: 2.2428
Epoch [11/30], Step [400/600], d_loss: 0.0226, g_loss: 6.7286
Epoch [11/30], Step [600/600], d_loss: 0.3276, g_loss: 5.9649
Epoch [12/30], Step [200/600], d_loss: 0.1900, g_loss: 3.3208
Epoch [12/30], Step [400/600], d_loss: 0.1364, g_loss: 4.3528
Epoch [12/30], Step [600/600], d_loss: 0.1535, g_loss: 5.0176
Epoch [13/30], Step [200/600], d_loss: 0.0690, g_loss: 5.6192
Epoch [13/30], Step [400/600], d_loss: 0.4900, g_loss: 6.0616
Epoch [13/30], Step [600/600], d_loss: 0.2419, g_loss: 5.0045
Epoch [14/30], Step [200/600], d_loss: 0.1356, g_loss: 6.1639
Epoch [14/30], Step [400/600], d_loss: 0.2905, g_loss: 5.1418
Epoch [14/30], Step [600/600], d_loss: 0.1162, g_loss: 3.8851
Epoch [15/30], Step [200/600], d_loss: 0.1947, g_loss: 4.9186
Epoch [15/30], Step [400/600], d_loss: 0.4006, g_loss: 4.6810
Epoch [15/30], Step [600/600], d_loss: 0.1922, g_loss: 5.7188
Epoch [16/30], Step [200/600], d_loss: 0.1492, g_loss: 5.4390
Epoch [16/30], Step [400/600], d_loss: 0.1732, g_loss: 4.0585
Epoch [16/30], Step [600/600], d_loss: 0.1563, g_loss: 5.5340
Epoch [17/30], Step [200/600], d_loss: 0.2931, g_loss: 5.3705
Epoch [17/30], Step [400/600], d_loss: 0.0923, g_loss: 4.1365
Epoch [17/30], Step [600/600], d_loss: 0.2255, g_loss: 3.8578
Epoch [18/30], Step [200/600], d_loss: 0.1149, g_loss: 5.9811
Epoch [18/30], Step [400/600], d_loss: 0.2010, g_loss: 7.8371

```
Epoch [18/30], Step [600/600], d_loss: 0.1779, g_loss: 6.5284
Epoch [19/30], Step [200/600], d_loss: 0.2664, g_loss: 4.4013
Epoch [19/30], Step [400/600], d_loss: 0.2816, g_loss: 3.9989
Epoch [19/30], Step [600/600], d_loss: 0.2190, g_loss: 5.6114
Epoch [20/30], Step [200/600], d_loss: 0.1454, g_loss: 5.7738
Epoch [20/30], Step [400/600], d_loss: 0.4151, g_loss: 4.2423
Epoch [20/30], Step [600/600], d_loss: 0.2563, g_loss: 5.3545
Epoch [21/30], Step [200/600], d_loss: 0.2400, g_loss: 3.9080
Epoch [21/30], Step [400/600], d_loss: 0.2972, g_loss: 3.1066
Epoch [21/30], Step [600/600], d_loss: 0.3191, g_loss: 4.9085
Epoch [22/30], Step [200/600], d_loss: 0.4887, g_loss: 2.7655
Epoch [22/30], Step [400/600], d_loss: 0.3913, g_loss: 5.9228
Epoch [22/30], Step [600/600], d_loss: 0.3821, g_loss: 4.1024
Epoch [23/30], Step [200/600], d_loss: 0.2842, g_loss: 5.6577
Epoch [23/30], Step [400/600], d_loss: 0.1909, g_loss: 5.4785
Epoch [23/30], Step [600/600], d_loss: 0.5301, g_loss: 3.9979
Epoch [24/30], Step [200/600], d_loss: 0.4222, g_loss: 3.2816
Epoch [24/30], Step [400/600], d_loss: 0.3890, g_loss: 5.1864
Epoch [24/30], Step [600/600], d_loss: 0.4407, g_loss: 4.0085
Epoch [25/30], Step [200/600], d_loss: 0.5739, g_loss: 4.1395
Epoch [25/30], Step [400/600], d_loss: 0.1865, g_loss: 4.3033
Epoch [25/30], Step [600/600], d_loss: 0.3071, g_loss: 4.6733
Epoch [26/30], Step [200/600], d_loss: 0.3400, g_loss: 4.7516
Epoch [26/30], Step [400/600], d_loss: 0.2576, g_loss: 4.2740
Epoch [26/30], Step [600/600], d_loss: 0.2202, g_loss: 4.1424
Epoch [27/30], Step [200/600], d_loss: 0.1603, g_loss: 5.7559
Epoch [27/30], Step [400/600], d_loss: 0.3114, g_loss: 5.6907
Epoch [27/30], Step [600/600], d_loss: 0.4424, g_loss: 3.8533
Epoch [28/30], Step [200/600], d_loss: 0.2447, g_loss: 3.7414
Epoch [28/30], Step [400/600], d_loss: 0.2269, g_loss: 4.1745
Epoch [28/30], Step [600/600], d_loss: 0.4053, g_loss: 3.4032
Epoch [29/30], Step [200/600], d_loss: 0.4679, g_loss: 3.4124
Epoch [29/30], Step [400/600], d_loss: 0.3794, g_loss: 3.5461
Epoch [29/30], Step [600/600], d_loss: 0.2556, g_loss: 3.2588
Epoch [30/30], Step [200/600], d_loss: 0.3073, g_loss: 4.0655
Epoch [30/30], Step [400/600], d_loss: 0.2802, g_loss: 4.5550
Epoch [30/30], Step [600/600], d_loss: 0.4073, g_loss: 3.8957
```

```
[17]
✓ Os
z = torch.randn(batch_size, latent_size).to(device)
fake_images = G(z)

fake_images = fake_images.reshape(fake_images.size(0), 1, 28, 28)
save_image(fake_images, 'final_generated_images.png')

[18]
✓ Os
import matplotlib.pyplot as plt
import numpy as np

grid = torchvision.utils.make_grid(fake_images[:64], nrow=8, normalize=True)
npimg = grid.cpu().numpy()

plt.figure(figsize=(8,8))
plt.imshow(np.transpose(npimg, (1,2,0)))
plt.axis('off')
plt.show()
```



```
import torch

import torch.nn as nn

import torchvision

import torchvision.transforms as transforms

import torchvision.utils as vutils

import matplotlib.pyplot as plt

import os


# Device configuration

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# Create output directory

os.makedirs("images", exist_ok=True)


# Hyperparameters

batch_size = 128

latent_size = 100

hidden_size = 256

image_size = 784

num_epochs = 30

learning_rate = 0.0002


# MNIST dataset

transform = transforms.Compose([

    transforms.ToTensor(),

    transforms.Normalize(mean=(0.5,), std=(0.5,))

])
```

```
dataset = torchvision.datasets.MNIST(root='./data',  
                                     train=True,  
                                     transform=transform,  
                                     download=True)  
  
data_loader = torch.utils.data.DataLoader(dataset=dataset,  
                                           batch_size=batch_size,  
                                           shuffle=True)
```

Discriminator

```
class Discriminator(nn.Module):  
  
    def __init__(self):  
        super(Discriminator, self).__init__()  
  
        self.model = nn.Sequential(  
            nn.Linear(image_size, hidden_size),  
            nn.LeakyReLU(0.2),  
            nn.Linear(hidden_size, hidden_size),  
            nn.LeakyReLU(0.2),  
            nn.Linear(hidden_size, 1),  
            nn.Sigmoid()  
        )  
  
    def forward(self, x):  
        return self.model(x)
```

Generator

```
class Generator(nn.Module):  
  
    def __init__(self):
```

```

super(Generator, self).__init__()

self.model = nn.Sequential(

    nn.Linear(latent_size, hidden_size),

    nn.ReLU(),

    nn.Linear(hidden_size, hidden_size),

    nn.ReLU(),

    nn.Linear(hidden_size, image_size),

    nn.Tanh()

)


def forward(self, x):

    return self.model(x)


D = Discriminator().to(device)
G = Generator().to(device)


criterion = nn.BCELoss()

d_optimizer = torch.optim.Adam(D.parameters(), lr=learning_rate)
g_optimizer = torch.optim.Adam(G.parameters(), lr=learning_rate)


# Training

total_step = len(data_loader)


for epoch in range(num_epochs):

    for i, (images, _) in enumerate(data_loader):

        images = images.reshape(batch_size, -1).to(device)

```



```
real_labels = torch.ones(batch_size, 1).to(device)
fake_labels = torch.zeros(batch_size, 1).to(device)
```

```
# Train Discriminator
```

```
outputs = D(images)
d_loss_real = criterion(outputs, real_labels)
real_score = outputs
```

```
z = torch.randn(batch_size, latent_size).to(device)
fake_images = G(z)
outputs = D(fake_images.detach())
d_loss_fake = criterion(outputs, fake_labels)
```

```
d_loss = d_loss_real + d_loss_fake
```

```
d_optimizer.zero_grad()
d_loss.backward()
d_optimizer.step()
```

```
# Train Generator
```

```
z = torch.randn(batch_size, latent_size).to(device)
fake_images = G(z)
outputs = D(fake_images)
g_loss = criterion(outputs, real_labels)
```

```
g_optimizer.zero_grad()
g_loss.backward()
g_optimizer.step()
```

```

if (i+1) % 200 == 0:

    print(f"Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{total_step}], "
          f"d_loss: {d_loss.item():.4f}, g_loss: {g_loss.item():.4f}")

# Save sample images

with torch.no_grad():

    z = torch.randn(64, latent_size).to(device)

    fake_images = G(z).reshape(-1, 1, 28, 28)

    vutils.save_image(fake_images, f'images/fake_images-{epoch+1}.png', normalize=True)

# Generate final images

with torch.no_grad():

    z = torch.randn(64, latent_size).to(device)

    fake_images = G(z).reshape(-1, 1, 28, 28)

    vutils.save_image(fake_images, 'final_generated_images.png', normalize=True)

# Display final generated grid

img = torchvision.utils.make_grid(fake_images.cpu(), nrow=8, normalize=True)

plt.figure(figsize=(6,6))

plt.imshow(img.permute(1, 2, 0))

plt.axis('off')

plt.show()

```